© 2024. This manuscript version is made available under the CC-BY-NC-ND 4.0 license https://creativecommons.org/licenses/by-nc-nd/4.0/. The final version was published in Expert Systems with Applications 245: 123030, https://doi.org/10.1016/j.eswa.2023.123030

Prov-Dominoes: An Approach for Knowledge Discovery from Provenance Data

Victor Alencar[1,2]

Brazilian Navy, CASNAV, Rio de Janeiro, RJ, Brazil.

Troy Kohwalter[2]

UFF, Instituto de Computação, Niterói, RJ, Brazil.

Vanessa Braganholo[2]

UFF, Instituto de Computação, Niterói, RJ, Brazil.

José Ricardo da Silva Junior[3,*]

IFRJ, Departamento de Computação, RJ, Brazil.

Leonardo Murta[2]

UFF, Instituto de Computação, Niterói, RJ, Brazil.

Abstract

Provenance has become increasingly relevant to understanding, auditing, and reproducing computational tasks. The provenance analysis processes can often be overwhelming to the user due to the large volume of data, the multiple relationships among data, and the implicit information buried in the data. Existing provenance analysis tools use either visual exploration (which is overwhelming for large provenance graphs) or do not support the exploration of implicit provenance data, such as the inferences of the PROV Data Model Constraints. To fill in this gap, we introduce Prov-Dominoes, a tool designed to interactively

*Corresponding author.

Email addresses: victor.alencar@marinha.mil.br (Victor Alencar), tkohwalter@ic.uff.br (Troy Kohwalter), vanessa@ic.uff.br (Vanessa Braganholo), jose.junior@ifrj.edu.br (José Ricardo da Silva Junior, leomurta@ic.uff.br (Leonardo Murta)

[1]Centro de Análises de Sistemas Navais.

[2]Universidade Federal Fluminense.

[3]Instituto Federal do Rio de Janeiro.

enable knowledge discovery on provenance data. Prov-Dominoes promotes the provenance relationships among entities, activities, and agents into first-class elements represented by domino tiles. It allows users to combine and compose such domino tiles visually and interactively, using GPU. The benefits of Prov-Dominoes are three-fold: first, it uses matrices to display provenance data, which is more compact than graphs; second, it allows users to easily explore implicit information; third, it is capable of efficiently processing large datasets using GPUs. We evaluated Prov-Dominoes over distinct case studies, allowing the observation of Prov-Dominoes in action. We also evaluated the performance of sequential combinations executed in Prov-Dominoes when dealing with provenance data with thousands of relations, contrasting their executions in GPU and CPU. The results showed that, for a large dataset, GPU was more than a hundred times faster than CPU.

Keywords: knowledge discovery, data analysis, provenance, gpu computing
2020 MSC: 68U35, 68-04, 68-11

1. Introduction

The problem of systematically capturing and managing the provenance[4] for computational tasks has increasingly received attention because of its relevance to a wide range of domains and applications (Freire et al., 2008), such as bioinformatics, astronomy, and engineering. Without provenance, it is difficult to reproduce and share results, solve problems collaboratively, validate results with different input data, and understand the process used to solve a particular problem. Additionally, data products' longevity becomes limited without precise and sufficient information about its generation process, which tends to diminish its value significantly (Silva et al., 2007).

The provenance of objects (digital or not) can be represented by a Directed Acyclic Graph (DAG) of causality, enriched with annotations (Moreau et al.,

[4]Information about entities, activities, and people involved in producing a piece of data or thing (Missier & Moreau, 2013).

2008a). This type of graph is useful for understanding the process of data generation. In the graph, the edges point to the past, indicating what has been done so far. It is worth noting that the provenance graph differs from a data or control flow graph, where the edges point to what will be executed in the future.

During provenance understanding, the researcher must make explicit any implicit information that could otherwise be inferred only from context (Newbury & Lippincott, 2019). Although the visualization of provenance in the form of graphs is common, such structures can be limited when there is a need to combine data to uncover new information. Besides, provenance graphs can be composed of thousands of vertices and edges, revealing challenges such as the competence to obtain holistic perspectives over large provenance data. Moreover, when performing explorations on large provenance graphs, navigation on these large graphic structures becomes less feasible, opening an avenue for more concise ways of visualization analysis.

Most of the existing provenance tools to assist provenance understanding (Kohwalter et al., 2016; Ellson et al., 2004; Bastian et al., 2009; Del Rio & Da Silva, 2007; Macko & Seltzer, 2011) are based on node-link diagrams, relying on visualization features over edges and vertices for their comprehension. Other tools, such as VisTrails (Callahan et al., 2006) and Taverna (Hull et al., 2006), provide infrastructure for data exploration and visualization through workflows. Both workflows and node-link diagrams do not support data combinations. For instance, they do not allow the derivation of implicit provenance data, such as constraints (Cheney et al., 2013c). By unveiling implicit data, new analysis possibilities arise. Moreover, they do not scale well for large provenance data, where the navigation over graphs becomes unfeasible. In fact, using graphs to manage a huge amount of data can be challenging in terms of visualization and exploration (Burch et al., 2021).

Aiming to fill in this gap, in this paper, we introduce Prov-Dominoes, a new approach for provenance understanding and knowledge discovery. Prov-Dominoes consume provenance data that is represented in PROV, the W3C

standard provenance model (Missier & Moreau, 2013) and allows for their interactive exploration through the dominoes game's metaphor. Each existing relationship in the provenance data among entities, activities, and agents (PROV basic constructs) is represented by a basic domino tile, which can be visually and interactively combined with other existing tiles to produce derived tiles. For instance, users can combine a basic tile that contains the entities used by activities with another basic tile that contains the entities generated by activities, producing a derived tile that contains the entities derived from other entities. These combinations can be used to uncover implicit relationships in the data.

Under the hood, Prov-Dominoes represents domino tiles as matrices, as they allow fast data combination and concise visualizations. According to Wu et al. (2008), "matrix visualization is a graphical technique that can simultaneously explore the associations of up to thousands of subjects, variables, and their interactions, without first reducing dimension". When analyzing large provenance graphs, matrix visualization allows a more concise visualization of the relationships on the graph without extensive navigation on a large graph structure. Additionally, to be able to deal with large data sets and still offer a good user interaction experience, Prov-Dominoes uses GPU. This way, exploring large amounts of data can be done without compromising the user's interaction with the system in terms of response time.

In order to evaluate the effectiveness and efficiency of our approach, we aim to answer the following research questions:

RQ1 *How effective is Prov-Dominoes in supporting interactive knowledge discovery on provenance data?*

- RQ1.1 How Prov-Dominoes uncovers implicit information?
- RQ1.2 How Prov-Dominoes provides a holistic perspective?
- RQ1.3 How Prov-Dominoes supports concise analysis?

RQ2 *How efficient is Prov-Dominoes when running in GPU in comparison to CPU?*

To answer these questions, we evaluated Prov-Dominoes over three distinct case studies. The first case study uses provenance from a VisTrails' workflow sample called "Head", while the second uses provenance captured from a smart home service. Finally, the last case study relies on large provenance data collected from Twitter to contrast GPU and CPU performances.

The remaining of this paper is organized as follows: Section 2 provides some background about provenance and introduces a guiding scenario that is explored throughout the following sections. Section 3 introduces Prov-Dominoes. In Section 4, we present three different case studies to evaluate our tool and discuss the results of the explorations performed on such case studies. Section 5 presents the related work and highlights how Prov-Dominoes distinguishes itself from other provenance tools. Section 6 concludes this work, presenting the contributions, limitations, and future work.

2. Background

Provenance is well understood in the context of art or digital libraries, where it respectively refers to the documented history of an art object or the documentation of processes in a digital object's life cycle (Committee et al., 2008). In 2006, at the International Provenance and Annotation Workshop (IPAW) (Groth et al., 2006), the participants were interested in data provenance, documentation, derivation, and annotation. As a result of the Provenance Challenges (Moreau et al., 2008b) presented at IPAW, the Open Provenance Model (OPM) (Moreau et al., 2011) was created. In 2013, another provenance model was developed as a W3C (Berners-Lee et al., 1994) effort, named PROV (Missier & Moreau, 2013), which can be viewed as the successor of OPM.

PROV has defined a provenance data model, called PROV-DM (Belhajjame et al., 2013), to support the interoperability and interchange of provenance in heterogeneous environments such as the web. At its core, PROV-DM describes the use and production of entities by activities, which may be influenced in various ways by agents (Belhajjame et al., 2013). According to PROV-DM,

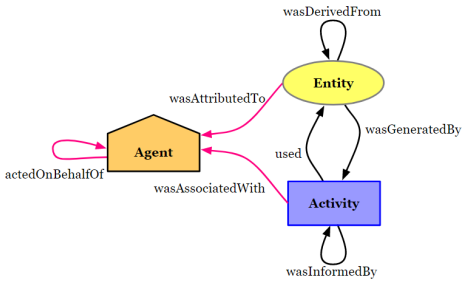


Figure 1: PROV-DM Types and Relations. Figure taken from Closa et al. (2017).

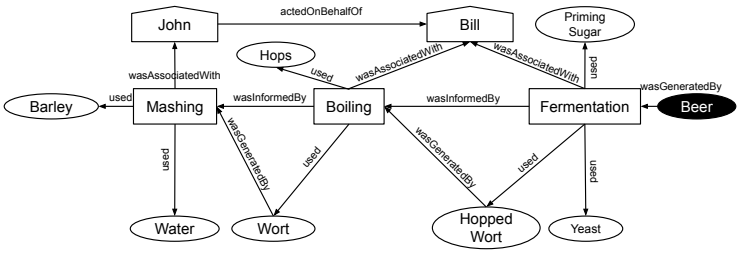


Figure 2: Provenance graph of beer production according to PROV-DM notation.

entities, activities, and agents are PROV-DM Types. Such types relate to each other through PROV-DM Relations, as shown in Figure 1. PROV-DM defines both types and relations as PROV-DM Concepts and represents provenance using such provenance concepts.

In this section, we introduce the fundamental aspects of PROV-DM, considered crucial to this work. The Subsections 2.2 and 2.3, respectively, refer to the types and relations concepts of provenance. This discussion is supported by a guiding example, detailed in Section 2.1.

2.1. Guiding Example

The concepts that will be used throughout this paper consider that *John* (an *agent*) is learning with *Bill* (also an *agent*) how to make *Beer* (an *entity*). In Figure 2, we show a provenance graph where *Bill* is responsible for *Boiling* and *Fermentation* (both *activities*) and delegates *Mashing* (another *activity*) to *John*, in order to produce the main *entity*, *Beer* (highlighted in black).

Listing 1: PROV-N fragment related to parts of Figure 2.

```
   ...
 1 activity(Mashing)
 2 activity(Boiling)
 3 activity(Fermentation)
 4 entity(Barley, [prov:type="Grain"])
 5 entity(Wort, [prov:type="Liquid"])
 6 entity(Hopped_Wort, [prov:type="Liquid"])
 7 entity(Beer, [prov:type="Liquid"])
 8 wasGeneratedBy(Wort, Mashing)
 9 wasGeneratedBy(Hopped_Wort, Boiling)
10 wasGeneratedBy(Beer, Fermentation)
11 agent(Bill)
   ...
```

Provenance graphs, such as the one in Figure 2, can be textually represented using the PROV Notation (PROV-N) (Cheney et al., 2013g), a syntax designed to write PROV-DM instances. The notation adopts a functional-style syntax consisting of a predicate name (referring to a PROV-DM Concept) and an ordered list of terms. The predicate and its terms compose a PROV-N expression. The expressions in the PROV-N fragment shown in Listing 1 represent some parts of the provenance graph in Figure 2.

2.2. PROV-DM Types

According to PROV-DM, "an *entity* (E) is a physical, digital, conceptual item or anything with some fixed aspects" (Belhajjame et al., 2013). An *activity* (Ac) "is something that occurs over a period of time and acts upon or with entities" (Belhajjame et al., 2013). An *agent* (Ag) "is something that bears some form of responsibility for an *activity* taking place, for the existence of an *entity*, or for another *agent*'s *activity*" (Belhajjame et al., 2013). In the example from Figure 2, *Barley* is an *entity*, *Mashing* is an *activity*, and *Bill* is an *agent*.

Additionally, to each *entity*, *activity*, or *agent*, PROV-N provides an expression to define a set of attribute-value pairs, as in line seven of Listing 1. One of

such attributes, the *"prov:type"*, provides typing capabilities: *Beer* is defined as an *entity* of type "Liquid".

2.3. PROV-DM Relations

According to PROV-DM, there are seven core relations of provenance: generation (*wasGeneratedBy*), usage (*used*), communication (*wasInformedBy*), derivation (*wasDerivedFrom*), association (*wasAssociatedWith*), attribution (*wasAttributedTo*), and delegation (*actedOnBehalfOf*). Two of these relations, *wasGeneratedBy* and *used*, represent explicit interactions between activities and entities. The *wasGeneratedBy* relation indicates the end of the production of an *entity* by an *activity*. Such an *entity* "did not exist before the generation and becomes available for use after the generation" (Belhajjame et al., 2013). The effect of ending production is instantaneous (Belhajjame et al., 2013). In the guiding example in Figure 2, *Wort wasGeneratedBy Mashing*. The *used* relation indicates the use of an *entity* by an *activity*. The effect of this use is also instantaneous (Belhajjame et al., 2013). In our guiding example, *Mashing* used *Water*.

Two other relations, *wasInformedBy* and *wasDerivedFrom*, represent implicit interactions between activities and entities. The *wasInformedBy* relation indicates the exchange of unspecified entities between two activities, a_1 and a_2, where a_2 uses an *entity* generated by a_1. Such communication implies that a_2 depends on a_1, since a_2 requires an *entity* generated by a_1 (Belhajjame et al., 2013). In the guiding example from Figure 2, *Boiling wasInformedBy Mashing*, since the *Boiling activity* uses the wort generated by the *Mashing activity*. The *wasDerivedFrom* relation is the transformation of an *entity* into another *entity*, an update of an *entity* resulting in another *entity*, or the construction of a new *entity* based on pre-existing entities (Belhajjame et al., 2013). In our guiding example, we see that *Mashing used Water* and *Wort wasGeneratedBy Mashing*. Although there is no explicit expression indicating that the *Wort entity* was derived from the use of *Water*, by inference (Cheney et al., 2013e) we could enrich the graph indicating that *Wort wasDerivedFrom Water*.

Table 1: "PROV-DM Relations at a Glance" (Missier et al., 2013) with trigrams.

Relation	**Subject**	**Object**	**Trigram**
wasGeneratedBy	Entity	Activity	WGB
used	Activity	Entity	USD
wasDerivedFrom	Entity	Entity	WDF
wasAssociatedWith	Activity	Agent	WAW
wasAttributedTo	Entity	Agent	WAT
wasInformedBy	Activity	Activity	WIB
actedOnBehalfOf	Agent	Agent	AOB

Three relations, *wasAssociatedWith*, *wasAttributedTo*, and *actedOnBehalfOf*, represent interactions that involve agents. The *wasAssociatedWith* relation indicates that an *agent* played a role in an *activity* (Belhajjame et al., 2013). In the guiding example, *Fermentation wasAssociatedWith Bill*. The *wasAttributedTo* relation indicates that an *entity* was generated by some *activity* associated with a given *agent*. Such a relation is useful when the *activity* is unknown or irrelevant (Belhajjame et al., 2013). In our example, we could enrich the graph indicating that *Wort wasAttributedTo John*, because *John* is associated with the *activity* that generated the *Wort*. The *actedOnBehalfOf* relation indicates the assignment of authority and responsibility for an *agent* to carry out a specific *activity* as a delegate or representative of the consigning *agent*. The consigning *agent* has some responsibility for the *activity* carried out by his delegate or representative (Belhajjame et al., 2013). In our guiding example, *John actedOnBehalfOf Bill*.

Summarizing, while not all PROV-DM Relations are binary, they all involve two primary elements, referred to as subject and object (Belhajjame et al., 2013). Table 2.3 indexes the seven core relations according to their two primary elements (Missier et al., 2013).

3. Prov-Dominoes

In this section, we present Prov-Dominoes[5], our exploratory analysis tool for provenance data. Prov-Dominoes is compatible with the PROV-N notation, allowing its adoption in different domains and applications. This section details the dominoes game metaphor, its possibilities, and the Prov-Dominoes' architecture. The first four subsections show how Prov-Dominoes represents PROV-DM Concepts and how they can be visualized, combined, and transformed. Finally, the last section presents the technologies and architectural components of the tool.

3.1. Dominoes Game Metaphor

We took the dominoes game as an inspiration to represent the elements in a provenance graph. Examining a provenance graph, we observe that the vertices are PROV-DM Types and the edges PROV-DM Relations. Thus, we took into account the following facts when designing Prov-Dominoes:

- The connections in a provenance graph are directed, associating a subject PROV-DM Type (e.g., *Mashing*) to an object PROV-DM Type (*e.g., Water*); and
- Each connection encloses a PROV-DM Relation (e.g., *Mashing* ***used*** *Water*, in Figure 2).

Consequently, each domino tile in Prov-Dominoes represents a PROV-DM Relation with the subject and object of the relation in the left and right faces, respectively. Then, we nominate such a domino tile with an uppercase trigram (three capital letters) that indicates which PROV-DM Relation the tile is representing. The trigrams are based on the main letters of the relation names defined by PROV-DM, thus enabling rapid identification of the underlying PROV-DM Relation (e.g., USeD→USD, as shown in the "trigram" column of Table 2.3).

[5]Prov-Dominoes is available at `https://gems-uff.github.io/prov-dominoes`.

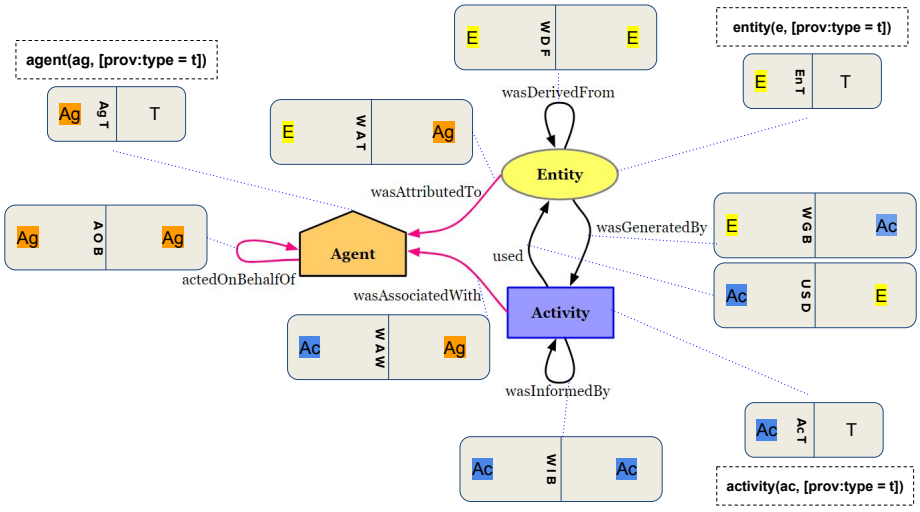


Figure 3: Core and Type domino tiles.

Finally, we refer to the tiles through their trigrams and subject-object faces (e.g., WGB[E|Ac] represents a relation *wasGeneratedBy*, which associates *entities* with *activities*).

The seven core PROV-DM relations were shaped as seven **core domino tiles**. Additionally, we developed three provenances **type domino tiles** to represent types of *agents*, *activities*, and *entities* expressed by the *"prov:type"* attribute, one for each PROV-DM Type: *agent-type* [Ag|T], *activity-type* [Ac|T], and *entity type* [E|T]. For instance, the fourth line of Listing 1 indicates that the *entity (E) Barley* is of type (T) *Grain*. Domino's tiles core and type are depicted in Figure 3.

Each core domino tile is represented as a matrix associated with a PROV-DM Relation. As each PROV-DM Relation has an expression in PROV-N, whose predicate name is the relation's name, the matrix can be built based on the PROV-N expressions associated with the PROV-DM Relation. PROV-N expressions have two required terms for representing relations. Thus, porting to a matrix representation, the first required term composes a row, and the second required term composes a column.

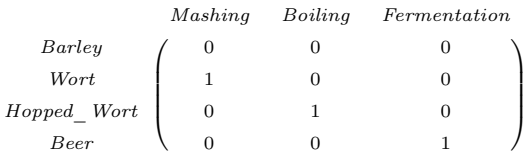


Figure 4: Matrix enclosing the *wasGeneratedBy* expressions from Listing 1.

For instance, in Listing 1, the expression in the eighth line has the predicate *wasGeneratedBy* (referring to the generation relation) and two required terms (*Wort* and *Mashing*). The first required term (*Wort*) becomes a row index in the WGB matrix, representing an instance of *entity* (E, subject of the generation relation). The second required term (*Mashing*) becomes a column index in the WGB matrix, representing an instance of *activity* (Ac, object of the generation relation). The cell $\mathrm{WGB}_{i,j}$ of the matrix is set by counting the number of expressions with the same pair of required terms. In this case: $\mathrm{WGB}_{Wort,Mashing} = 1$. The tenth line has the same *wasGeneratedBy* predicate, then its pair of terms (*Beer* and *Fermentation*) sets $\mathrm{WGB}_{Beer,Fermentation} = 1$. The resulting matrix that encloses the *wasGeneratedBy* expressions in Listing 1 is shown in Figure 4.

In the case of domino tiles' type, the same idea of pair of indexes to map cells works, where the PROV-DM Type indexes the row, and the value of the defined "prov:type" indexes the column. For instance, in the fourth line of Listing 1, the cell $\mathrm{EnT}_{Barley,Grain}$ of the matrix EnT enclosing *entity-type* [E|T] is set to one. The resulting matrix that encloses the *entity-type* expressions in Listing 1 is shown in Figure 5.

3.2. Domino Tile Visualizations

Prov-Dominoes has two kinds of content visualization of the domino tiles besides basic line and bar graphics: matrix and centrality graph. They were chosen due to their concise visualization properties. The matrix visualization is a tabular representation of data, where the cells are colored according to their value. Grey indicates zero, blue indicates the maximum value present in

$$
\begin{array}{c} \\ Barley \\ Wort \\ Hopped_Wort \\ Beer \end{array}
\begin{array}{c} \begin{array}{cc} Grain & Liquid \end{array} \\ \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix} \end{array}
$$

Figure 5: Matrix enclosing *entity-type* expressions from Listing 1.

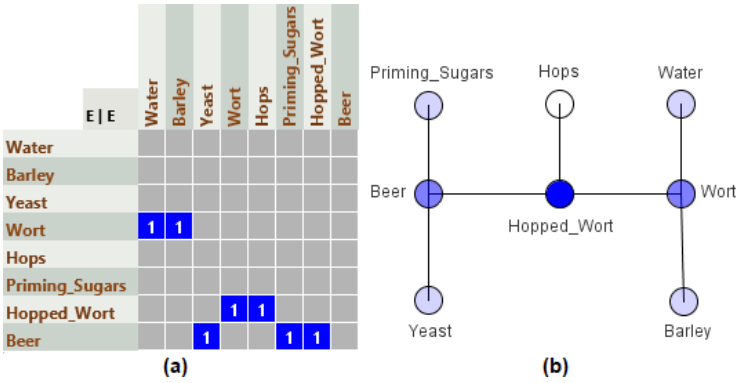


Figure 6: Matrix visualization of the *wasDerivedFrom* domino tile of the guiding example (a), and its centrality graph (b).

the matrix, and white indicates the minimum value different from zero. Values between maximum and minimum are colored in shades of blue (from blue to white).

For instance, consider the underlying matrix of the domino tile *wasDerivedFrom*, from our guiding example, shown in Figure 6.a. The blue cells illustrate the maximum value (1), indicating that *Wort* was derived from *Water* and *Barley*. Such matrix visualization allows a more concise identification of the relationships on the graph without extensive navigation on a large graph structure.

The centrality graph visualization is an *eigenvector centrality* implementation. Centrality measures indicate that some nodes are more important (central) than others in a graph. The idea of centrality was first introduced in the context of social systems, where the location of an individual in the network

may correlate with its influence or power in group processes (Crucitti et al., 2006). The *eigenvector centrality* (also called prestige score (Zaki et al., 2014)) measures such importance through recursive connections. A high eigenvector score means that a node is connected to many nodes with high scores (Newman, 2008). Because this centrality visualization requires a graph, it only makes sense for adjacency matrices encoded in double domino tiles (with the same subject-object faces). When analyzing the square matrix of a double domino tile, it is unclear which cells have more importance over others, as the cell's value only accounts for occurrence. The centrality graph provides scores for an important perspective.

For example, Figure 6.b exhibits the *eigenvector centrality* graph for the *wasDerivedFrom* domino tile of our guiding example. The nodes are colored in shades of blue, where the higher score node is blue and the lower white. The graph suggests *Hopped_ Wort* as the most important entity in beer production. Such specific centrality graphs focus on just one PROV-DM Type (e.g., entities, in Figure 6.b) and provide a shorter visualization compared to the whole provenance graph. Besides, the nodes' scores provide means to navigate such reduced graphs in a more consistent direction, following the high or low centrality scores.

3.3. Domino Tiles Combinations

In Prov-Dominoes, data combination is achieved by merging two domino tiles into one. Figure 7 illustrates the three existing types of combinations: by approaching part-equal[6] domino tiles horizontally (multiplication-merge) in the left or the right, as in Figure 7.a; by approaching two equal[7] domino tiles in the top (sum-merge), as in Figure 7.b; and by approaching two equal domino tiles in the bottom (subtraction-merge), as in Figure 7.c.

As multiplication-merge requires the merging faces to be the same, resembling the dominoes game when leaning two domino tiles, users may need to

[6] Only the faces and dimensions being approached should be equal.

[7] Equality of faces and dimensions.

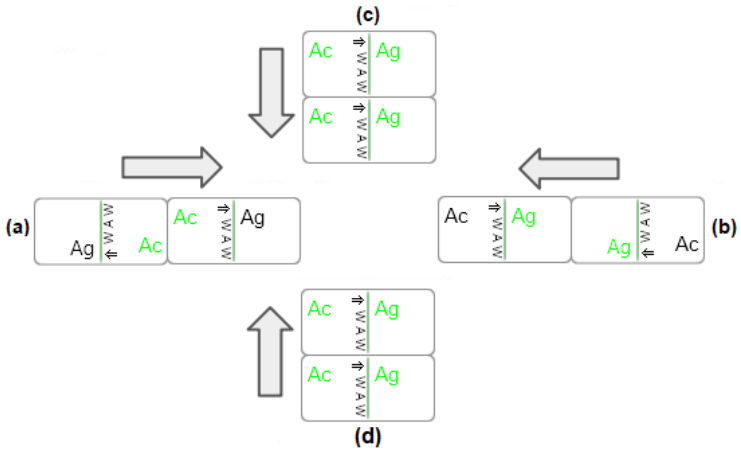


Figure 7: Domino tiles combinations through: the left (left-multiplication) (a); or the right (right-multiplication) (b); the top (sum) (c); and the bottom (subtraction) (d).

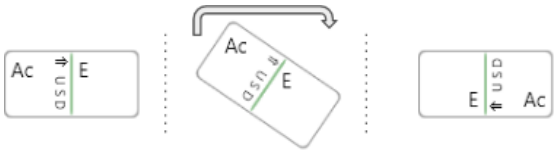


Figure 8: Transposition of a domino tile.

transpose (reverse the faces) the domino tile to match faces or benefit from other visualization perspectives. Figure 8 shows such domino tile transposition. The transposition process starts when the user double-clicks the domino, issuing a transposition of its matrix (e.g., USD $\rightarrow$ USD^T).

When approaching faces horizontally, Prov-Dominoes performs the multiplication of matrices, generating a new matrix for the resulting domino tile, as shown in Figure 9. The multiplication takes into account: PROV-DM Types match (object of the first domino tile matches the subject of the second) and dimension match (number of columns in the first domino tile matrix equals the number of rows of the second). Before the multiplication, Prov-Dominoes sorts the columns of the first matrix and the rows of the second. Such a sorting assures that all elements appear in the same order in both tiles for a correct multiplication.

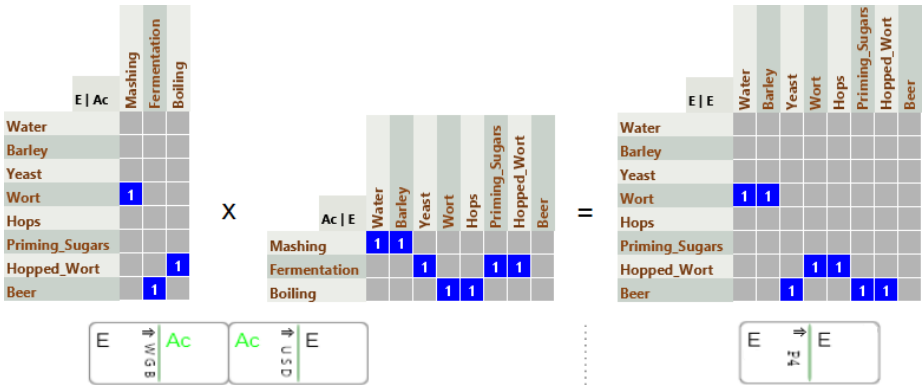


Figure 9: Domino tile combination: WGB [E|Ac] × USD [Ac|E], resulting in a WDF [E|E] domino tile.

The example shown in Figure 9 consists of a derivation inference (Cheney et al., 2013e). The domino tile on the left-hand side shows entities generated by activities (WGB [E|Ac]), and the domino tile on the right-hand side shows activities that used entities (USD [Ac|E]). When multiplying both by connecting the activity side, the generated domino tile shows entities derived from other entities (WDF [E|E]). In the rows of the matrix above the new domino tile, we can see that only the generated entities have some cells filled: *Hopped_ Wort*, *Wort*, and *Beer*. In the columns, we can see the entities that took part in the generation of the row entities: *Hopped_ Wort* (row) was derived from *Wort* (column) and *Hops* (column), *Wort* (row) was derived from *Water* (column) and *Barley* (column), and *Beer* (row) was derived from *Hopped_ Wort*, *Yeast*, and *Priming_ Sugars*. This allows analysis that would otherwise be difficult to make just by looking at the corresponding provenance graph.

Two other inferences of the PROV-DM Constraints can be obtained by combining the domino tiles. The combination of USD [Ac|E] and WGB [E|Ac] by approaching the E faces produces a domino tile corresponding to WIB [Ac|Ac] (wasInformedBy) (Cheney et al., 2013b). The combination of WGB [E|Ac] and WAW [Ac|Ag] (wasAssociatedWith) by approaching the Ac faces produces a domino tile corresponding to WAT [E|Ag] (wasAttributedTo) (Cheney et al.,

2013a). Once new data arise from combinations, portions of provenance are unveiled, providing a more holistic perspective for provenance analysis.

The sum-merge and subtraction-merge require the faces and dimensions of both domino tiles to be equal. While approaching the domino tiles, the faces' letters become green (merge conditions match) or red (merge conditions fail). When green, the domino tiles are merged, and a new domino tile arises with the same faces and dimensions as the previous two. Under the hood, Prov-Dominoes performs the sum (top-approach) or subtraction (bottom-approach) of the matrices. To ensure the match of rows and columns, Prov-Dominoes previously sorts rows and columns of both domino tiles.

3.4. Domino Tile Transformations

In this section, we describe the available transformations on Prov-Dominoes. These transformations affect only the matrix's content beneath a domino tile without affecting its external image. Such content-only transformations may reveal implicit information about the data. They are organized as follows:

- **Operations**: Transformations that affect cell values or matrix dimensions;
- **Filters**: Transformations that filter (by setting cell value to zero) cells according to some filtering rule; and
- **Sorting**: Transformations that rearrange cells by moving rows and columns.

In the list below, we briefly summarize the domino tile transformations available in Prov-Dominoes, except for *transitive closure*, which we detail in the following. We refer to $A_{i,j}$ as a cell with row index i and column index j of a generic matrix A that represents a given domino tile:

- *Binarize*: $A_{i,j} \leq 0 \Rightarrow A_{i,j} = 0$ and $A_{i,j} > 0 \Rightarrow A_{i,j} = 1$;
- *High-Pass Filter*: $A_{i,j} \leq v \Rightarrow A_{i,j} = 0$, where v is a cutoff value;
- *Low-Pass Filter*: $A_{i,j} \geq v \Rightarrow A_{i,j} = 0$, where v is a cutoff value;

- *Trim*: Eliminates empty rows and columns;
- *Aggregate Rows/Columns*: Reduces rows/columns to one dimension (sum of the rows/columns);
- *Word on Row/Column*: $A_{i,j} = 0$ if the row/column label of the cell does not match a word (or regular expression); and
- *Sort by Row/Column Group*: Rearranges cells by grouping together non-zero cells in a row/column. The bigger the group, the closer to the top (if a row group) or to the left (if a column group).

Transitive closure is an allowed operation for a double domino tile (i.e., tiles with the same element in both sides) enclosing an adjacency matrix. The *transitive closure* of an adjacency matrix is the reachability[8] matrix. Typically, generating the *transitive closure* is the process of creating the reachability matrix, where $A_{i,j} = 1$ if a path exists from vertex i to j and $A_{i,j} = 0$ otherwise. Our approach generates an adapted reachability matrix. Instead of setting one when there is a path between i and j, we set $1/n$, where n represents the number of steps necessary to get from i to j. This provides an intuitive visualization of node distances, as the shades of blue are lighter for nodes distant from each other. Moreover, we assume each vertex can reach itself in one step.

Consider the example from Figure 10, where Figure 10. a shows the matrix of the WDF [E|E] domino tile produced from our guiding example. Figure 10.b shows the same matrix after the *transitive closure* operation. In Figure 10.b, we can see that the last row (*Beer*) is transitively derived from all ingredients of the beer provenance, as expected. The blue cells indicate ingredients that were participating in the last step of the beer production. The cells in shades of blue indicate indirect influences on the beer production. For instance, because *Water* and *Barley* were the first ingredients in the beer production, they have

[8] In graph theory, reachability refers to the ability to get from one vertex to another within a graph.

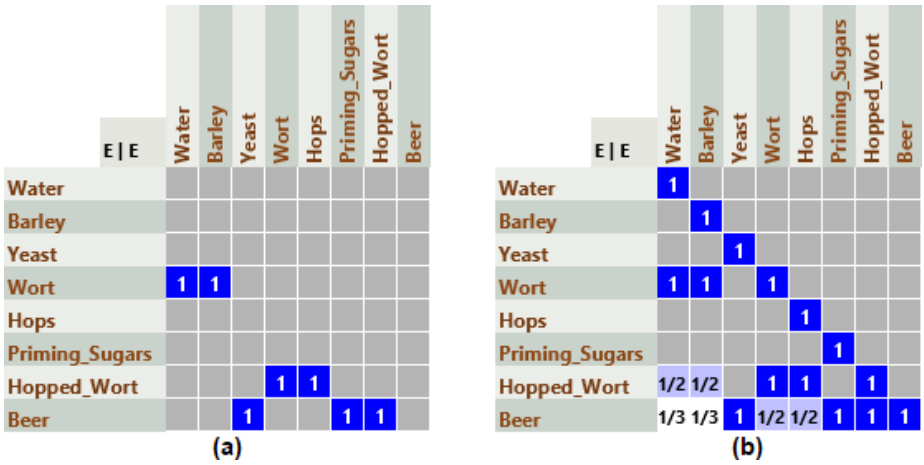


Figure 10: The *WDF* matrix visualization (a) and the same matrix after the *transitive closure* operation (b).

lighter blue tones. In fact, they are 3 steps distant to the *Beer entity*: (*Water*, *Barley*) → *Wort*, *Wort* → *Hopped_ Wort*, and *Hopped_ Wort* → *Beer*.

Furthermore, the *transitive closure* operation enables another inference of the PROV-DM Constraints, the delegation inference (Cheney et al., 2013d). For instance, in Figure 2, there is no connection between *Bill* (an *agent*) and *Mashing* (*an activity*). The delegation inference states that because *Bill* has one delegate (*John*), he is also associated with the activity (*Mashing*) of his delegate. If we combine WAW [Ac|Ag] with the *transitive closure* of AOB [Ag|Ag] (actedOnBehalfOf) by approaching the Ag faces, we obtain a new domino tile that is the same WAW [Ac|Ag], but considering the delegation inference. Thus, the cell of *Mashing* and *Bill*) is set to one.

3.5. Prov-Dominoes Architecture

The architecture of Prov-Dominoes has three layers: *data layer*, *processing layer*, and *presentation layer*. The *data layer* is responsible for reading the supported input files and parsing the data in the files to matrices for domino tiles assembling. The *processing layer* provides the algorithms for matrix combinations and transformations, both for GPU and CPU. These two layers are

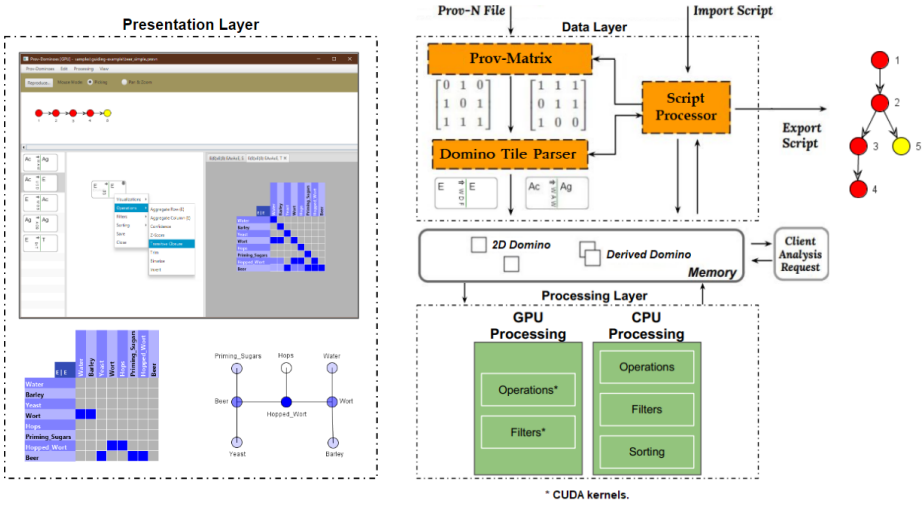


Figure 11: Prov-Dominoes' architecture.

depicted in Figure 11 and are described in this section.

The *data layer* has three inner components: *Prov-Matrix*, *Domino Tile Parser*, and *Script Processor*. The Prov-Matrix component is in charge of converting PROV-N expressions into matrices, one matrix per distinct PROV-DM Relation stated in the PROV-N file, as mentioned in Section 3.1. After the matrices are built, they are sent to the *Domino Tile Parser*. The *Domino Tile Parser* component is responsible for assembling the domino tile, both in terms of its matrix content and its non-graphical structures, such as the labels for the matrix and dimension types.

After this process, the domino tile data structure is available for the *presentation layer*, where the domino tiles are drawn and manipulated. The presentation layer also provides visualizations using centrality graphs as an alternative to the matrix visualization. Note that these graphs do not show the raw data. Instead, they show the eigenvector centrality, which can be applied to visualize adjacency matrices encoded by double domino tiles, as explained in Section 3.2. For visualizing the raw data in the graph format, users can adopt provenance visualization tools such as Prov Viewer (Kohwalter et al., 2016).

The *Script Processor* component is responsible for building and managing

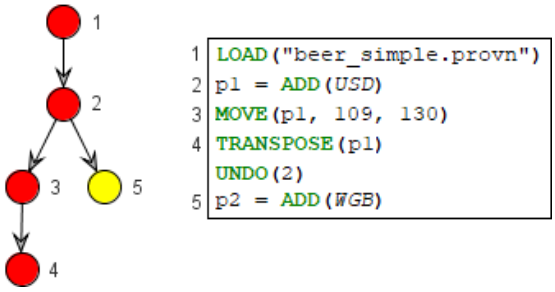


Figure 12: Exploration Provenance Script.

the history of commands (Domino tiles combinations or transformations) in a tree data structure. Such history functions as the provenance of the derived domino tiles, enabling the user to keep track and navigate through their explorations. The *Script Processor* enables the user to export the commands that led to the derived domino tiles (i.e., their provenance) to an Exploration Provenance Script (EPS). Similarly, the *Script Processor* allows the user to import the EPS to reproduce previous explorations.

In Figure 12, we show an example of an EPS. The first commands *LOAD*, *ADD*, *MOVE*, and *TRANSPOSE* are represented by nodes 1, 2, 3, and 4. The command of node 2 (*p1 = ADD(USD)*) has an identifier: "p1". The "p1" identifier indicates that the *ADD* command produced a domino tile (a copy of *USD* as a result of *ADD(USD)*) that was stored in "p1", allowing further commands to access such result by referring to "p1", as in the fourth command: *TRANSPOSE(p1)*. After the fourth command, the next command *UNDO(n = 2)* represents two undo performed in sequence, invalidating the n last commands and moving backward n nodes in the history tree. Commands performed after *UNDO* will imply a new branch created in the history tree, as in command *p2 = ADD(WGB)*, attributed to node 5.

The *processing layer* consists of two matrix processing components, CPU and GPU, that operate in a mutually exclusive processing mode. They are responsible for processing matrix-related transformations: operations, filters, and

sorting. The CPU processing component uses the open-source java library la4j[9] (Linear Algebra for Java). The library provides Linear Algebra primitives (matrices and vectors) and algorithms for Java. The GPU processing component is implemented in CUDA[10], a parallel computing platform and application programming interface (API) model created by NVIDIA. The coding in CUDA had as scope (asterisks in Figure 11) the following matrix-related transformations: addition, subtraction, transposition, multiplication, *transitive closure*, and binarization. The purpose of GPU usage is to enable faster processing when dealing with large provenance data. All transformations are executed in the CPU if GPU is not available in the computer.

4. Evaluation

In this section, we present distinct case studies to evaluate Prov-Dominoes, considering the following Research Questions:

RQ1: How effective is Prov-Dominoes in supporting interactive knowledge discovery on provenance data? As discussed in Section 1, making explicit any implicit provenance information is an aspect of provenance analysis. Thus, we considered the capability of uncovering implicit data (i.e., making it explicit) an effective aspect in supporting interactive knowledge discovery on provenance data. Moreover, as provenance data may be abundant in explicit information, we considered the capability of providing holistic perspectives over (possibly abundant) information another effective aspect to guide new discoveries. Additionally, we considered concise analysis as an aspect itself, whose benefits would integrate both strict (implicit and explicit) and holistic data, altogether compounding effective support for interactive data discovery. Thus, we broke down effectiveness into three sub-questions:

RQ1.1: How Prov-Dominoes uncovers implicit information? Provenance data may have implicit information buried into the data, hindering its

[9] `http://la4j.org`.

[10] `https://developer.nvidia.com/cuda-zone`.

perception. Uncovering such information may enrich the interactive knowledge discovery process.

RQ1.2: How Prov-Dominoes provides a holistic perspective? Similarly, big-picture analyses, provided by a holistic perspective of the data, may contribute to providing a wholesome comprehension of the data domain. We analyze if the explorations performed using Prov-Dominoes are capable of extracting such holistic perspectives.

RQ1.3: How Prov-Dominoes supports concise analysis? We assess if the matrix and *eigenvector centrality* graph visualizations provided by Prov-Dominoes are capable of integrating strict and holistic information, thus assisting the explorations. Furthermore, as discussed in Section 1, analyzing provenance graphs start to become an overwhelming task as the number of edges and vertices grows. Providing concise ways to visualize graphs may not only benefit the analysis on larger graphs, as discussed in Section 3.2, but also potentially assist pattern identifications regarding the arrangements of cells in a matrix. Moreover, the *eigenvector centrality* graph may benefit the analysis by per-type observations. For instance, first analyzing the centrality of agents, then entities, and then activities.

RQ2: How efficient is Prov-Dominoes when running in GPU in comparison to CPU? One of the aspects that can compromise the user experience while using software is the time they have to wait for a response after submitting a request for a computer system. Bouch et al. (2000) found that users tend to perceive systems that take longer than 10 seconds to process a web request as *suboptimal* systems. Furthermore, the user's tolerance for latency varies depending on their perception of the complexity of the requested task. Given that time tolerance is influenced by the subjective perception of the system's complexity, the relationship between response time and user satisfaction remains unclear. In this sense, we analyzed how faster Prov-Dominoes is when it uses GPU when compared to when it uses CPU for processing thousands of relations in order to have an estimate of the improvement magnitude of time for maximizing user's satisfaction.

In this sense, we analyzed how faster is Prov-Dominoes when it uses in the GPU when compared to when it uses the CPU for processing thousands of relations in order to have an idea of the improvement it would provide to the user satisfaction measuremagnitude of time for maximizing user's satisfaction.

In the following subsections, we describe the corpus and discuss the results of our explorations, first for the effectiveness evaluation (**RQ1**), then for the efficiency evaluation (**RQ2**).

4.1. Effectiveness

In this section, we present three distinct case studies as an attempt to answer **RQ1.1**, **RQ1.2**, and **RQ1.3**. First, we describe the materials and methods of the case studies, and then we present and discuss the results. In Section 4.1.3, we summon some exploratory practices frequently observed in the case studies.

4.1.1. Materials and Methods

We selected two different case studies in order to observe in action how Prov-Dominoes supports exploratory analyses in different domains. For each case study, we designed exploratory tasks for information extraction, considering their domains.

The **first case study** uses provenance from a VisTrails' workflow called "Head." We wanted to lean on a provenance generated by a Workflow Management System (WfMS) to check if our explorations could contribute to a better understanding of the provenance. The exploratory tasks aim at understanding which activities and parameters were essential to the workflow execution results. For this case study, we adopted member checking by inviting the workflow's author to provide feedback about our explorations.

The chosen Head Workflow gathers data derived from "The Visible Human Project" (Ackerman, 1998), corresponding to a person's head, and renders its bones and skin as a volumetric image. When exporting the Head Workflow from VisTrails to PROV-N, apparently, only the executions of the last exploration are available. The last exploration in the sample is called "volume rendering,"

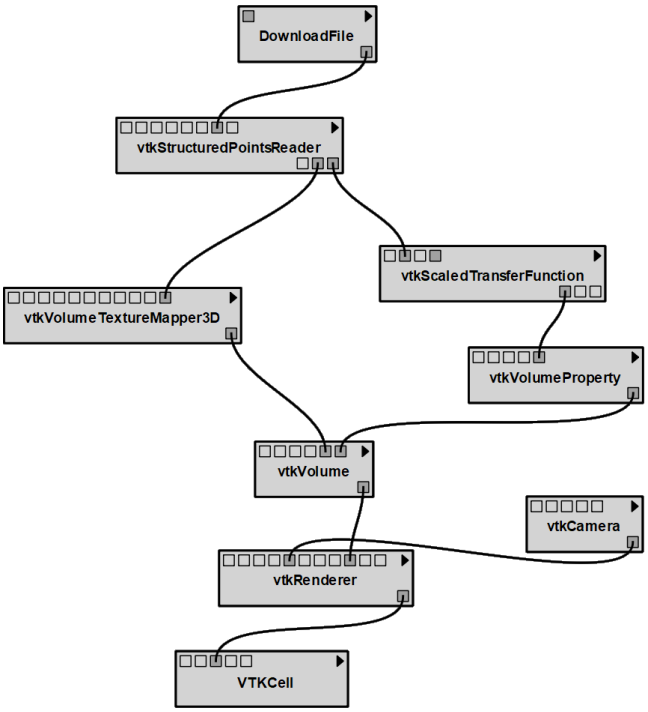


Figure 13: Head Workflow and its executed modules.

which is the one we considered for analysis. The modules used in the execution are based on the Visual Toolkit (VTK) (Schroeder et al., 2006) and depicted in Figure 13.

The exported provenance maps the modules as activities, the inputs and outputs as entities, and the executors of the workflow as agents. The provenance visualization of the workflow in VisTrails focuses on module (activities) connections, as shown in Figure 13. Each edge connecting two modules indicates that the output (*entity*) produced by the source module was used as input (*entity*) by the target module. However, we do not see such entities in Figure 13, as they are kept implicit. Furthermore, additional parameters one module may have are also kept implicit. As the "volume rendering" exploration was executed only by the workflow's author, we refrain from *agent* analysis in this case study.

The **second case study** uses provenance captured from a smart home ser-

vice called MiJia[11]. We chose such a case study as an example of the potential of provenance on smart devices and the Internet of Things. We downloaded the provenance captured by a user of such smart home service from ProvStore[12], a free service for storing, viewing, and collaborating on provenance data. The exploratory tasks investigate which activities are central to the smart home service's functioning and which agents are behind its operation.

The MiJia (Mi Home) Smart Home Service consists of a series of Xiaomi[13] smart devices operating together in a building. The smart devices communicate with a multi-functional gateway responsible for integrating them. The user can control and interact with smart devices through a MiJia app on her smartphone. The data generated by the service is recorded as PROV-N, where device data are represented as entities, commands issued to the devices as activities, and devices as agents. The captured provenance has expressions about four devices: a temperature sensor ("Sensor1"), a humidity sensor ("Sensor2"), and two unspecified devices ("Device1" and "Device2"), as shown in Figure 14. Additionally, there are two agent-types: *prov:SoftwareAgent* and *prov:Person*. The former encompasses the agents "Server" (operating on the cloud) and "Application" (operating on the user's smartphone), and the latter encompasses the agent "User".

4.1.2. Results and Discussion

In this section, we detail how transformations and visualizations are articulated together during exploratory tasks in each case study.

In the **first case study**, we decided to focus on entities (input parameters and outputs) to address the exploratory task of understanding which activities and parameters were essential to the workflow, as they represent connecting bounds between activities (modules). Thus, we explored derivation inference to relate entities to each other: WGB [E|Ac] × USD [Ac|E] = WDF [E|E] (Cheney

[11] `http://home.mi.com/index.html`.

[12] `https://openprovenance.org/store/documents/2148`.

[13] `https://www.mi.com`

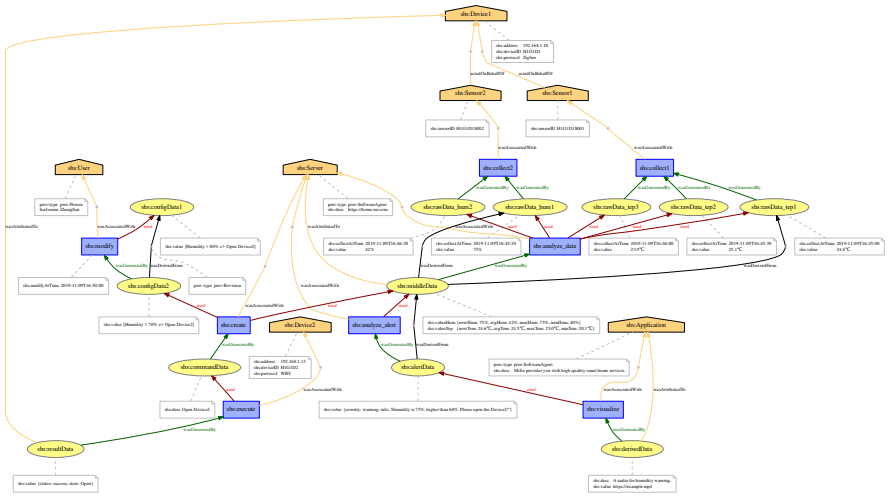


Figure 14: Provenance graph taken from ProvStore.

et al., 2013e). We then applied the *transitive closure* operation in the WDF domino tile to investigate dependencies amongst entities. We filtered the rows of the resulting domino tile to exhibit only outputs. Thus, in the rows, we have outputs, and in the columns, we have entities (inputs and outputs) that took part in the output (row) generation. Finally, we trimmed the domino tile to eliminate empty rows and columns. The EPS expressing the resulting domino tile is shown in Figure 15.a, and its resulting matrix is shown in Figure 15.b.

The two vertical lines in Figure 15.b indicate column entities that took part directly or indirectly in almost any output, except for the "vtkCamera" output. The left vertical line is the output of the download module, which gathers the dataset from the Visible Human Project. The right vertical line is the download URL parameter where the dataset is located. Such entities suggest two conditions without which the workflow could not proceed: internet connection and correctness of the URL.

The horizontal line in Figure 15.b refers to the most indirectly dependent output, which is the output of the "vtkRenderer." Such output depends on all inputs and intermediate outputs, making it sensitive to errors propagated

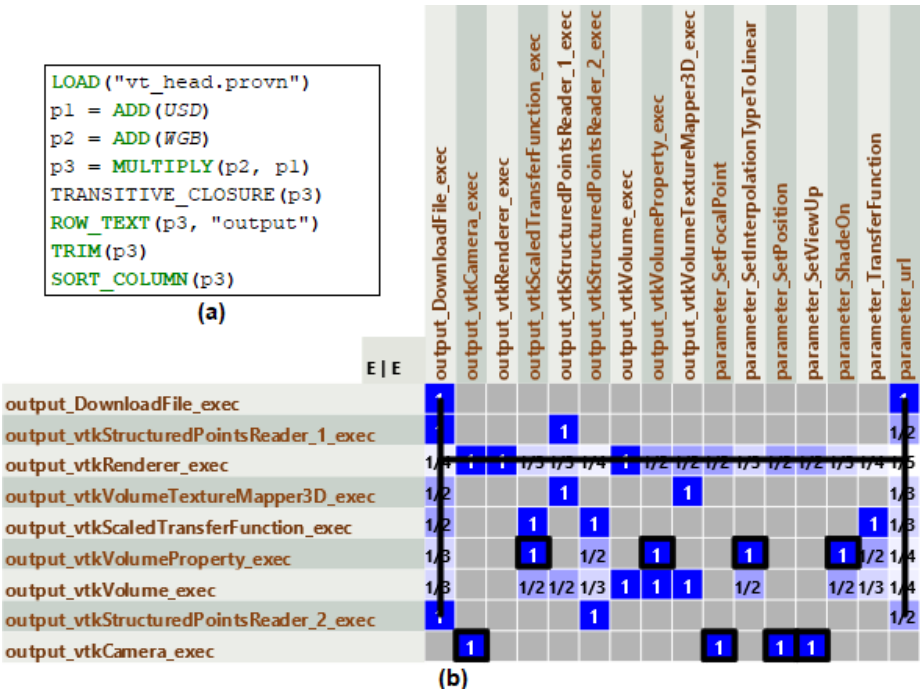


Figure 15: EPS of the analysis on entities (a); Resulting domino tile and matrix: outputs in the rows, and inputs and outputs in the columns. The columns represent entities that took part in the output (row) generation (b).

by other parameters or intermediate activities. Thus, errors or undesired results detected in the output of the "vtkRenderer" module are harder to debug. Such dependencies also suggest that the "vtkRenderer" module is an aggregation activity, the final destination to prior derived entities, where they may be assembled or processed together, potentially representing a macro aspect or goal of the workflow.

Moreover, in the matrix of Figure 15.b, we counted the rows with more direct dependencies (blue cells, cell value = 1), and two rows arose: the outputs of "vtkCamera" and "vtkVolumeProperty." Both rows have four direct dependencies. Direct dependency is a measure of coupling, useful to determine how complex the testing of various parts of a design is likely to be (Chidamber & Kemerer, 1994). According to Yadav and Khan (Yadav & Khan, 2010), cou-

pling increases the complexity of the system, which makes the system difficult to understand and maintain. Such coupling remarks suggest that "vtkCamera" and "vtkVolumeProperty" are activities that are complex to maintain and understand.

We interviewed the workflow's author and asked her to evaluate whether our analysis correctly highlights relevant parts of the workflow. The answer was the following: "*Yes because the analysis highlights the main components of the workflow. The "vtkRenderer" module is the one that draws the "Head". The module where I specify how the elements will be drawn is the "vtkVolumeProperty". The "vtkCamera" sets an observer on the scene and tells it where to look. Without the "vtkCamera", it would not be possible to see what the "vtkRenderer" module drew.*"

Then, we asked if the analysis was useful. The reply was: "*I considered it extremely useful, especially to see the parameters actually used by the modules and have information on how they are influencing the workflow's execution. In the workflow view of VisTrails, we can see only the modules. Also, in the matrix, it is easier to recognize coupled modules than in the workflow view. Even in this case, as it is a relatively simple workflow! Imagine one with more connections. It would be much harder to perceive this information in the workflow view!*"

RQ1.1. *How Prov-Dominoes uncovers implicit information?*
In the provenance, there was no data directly relating entities (inputs and outputs) to each other. Such relation was made explicit by the derivation inference: WGB [E|Ac] × USD [Ac|E] = WDF [E|E]. By unveiling *entity* derivations, further explorations on entity dependencies became possible.

RQ1.2. *How Prov-Dominoes provides a holistic perspective?*
As stated by the workflow's author, the analysis on the matrix of Figure 15 was capable of highlighting the main components of the workflow, contributing to a wholesome understanding of the workflow execution.

RQ1.3. *How Prov-Dominoes supports concise analysis?*
The concise matrix visualization used in our explorations enabled pattern identifications such as the horizontal and vertical lines indicated in Figure 15, easing the analysis. Moreover, the blue shades in the rows indicated direct dependencies, assisting the dependency analysis. Finally, as stated by the workflow's author, the visualization provided by Prov-Dominoes is more scalable for workflows with many connections than a plain workflow view.

In the **second case study**, we applied the communication inference to address the exploratory task of uncovering activities central to the functioning of the smart home service. The following matrix multiplication was used to obtain activities communicating to each other: USD [Ac|E] × WGB [E|Ac] = WIB [Ac|Ac] (Cheney et al., 2013b). In Figure 16.a, we see the resulting matrix of the WIB [Ac|Ac] domino tile. The activities in the rows (e.g., "visualize") occur after the activities in the column (e.g., "analyze_alert") when the corresponding cell is greater than 1. We exhibit an extended version of the domino tile WAW [Ac|Ag] in Figure 16.b to observe which devices are behind activities. The original WAW [Ac|Ag] had no activities associated with "Device1", however "Sensor1" and "Sensor2" are delegates of "Device1" as stated by the AOB [Ag|Ag] domino tile. By combining WAW [Ac|Ag] × AOB [Ag|Ag] = [Ac|Ag] we obtain the indirect activities associated with "Device1". Then, we add up the resulting domino tile to the original WAW [Ac|Ag] to produce the extended version in Figure 16.b. We can see that the only agents associated with more than one activity are "Device1" (two activities) and "Server" (three activities), which suggests that both are relevant agents.

Analyzing together both domino tiles of Figure 16, we realize that temperature ("Sensor1") and humidity ("Sensor2") were collected ("collect1" and "collect2" activities) by the sensors ("Sensor1" and "Sensor2" agents), as we can see in Figure 16.b. Then the collected information was analyzed ("collect1" and "collect2" inform "analyze_data" in Figure 16.a) by the "Server" ("analyze_data" is associated with "Server" in Figure 16.b). After the analyses of the collected infor-

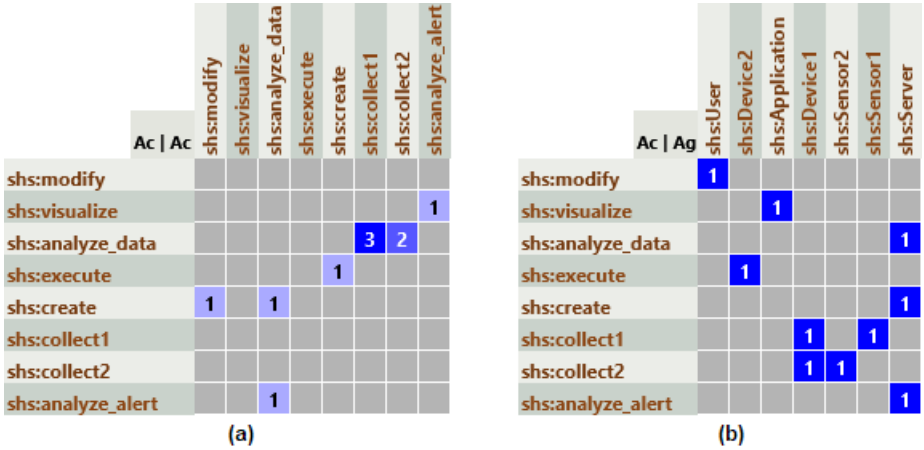


Figure 16: Activities communicating to each other (a); and Activities (rows) associated with agents (b).

mation from sensors, an alert analysis follows: "analyze_alert" was informed by "analyze_data" in Figure 16.a, also performed by the "Server" (in Figure 16.b, "analyze_alert' is associated with "Server"). Finally, the "visualize" activity ("visualize" was informed by "analyze_alert" in Figure 16.a) was executed by the "Application" agent ("Application" issues "visualize" in Figure 16.b), on the users' smartphone. To summarize: the "Server" analyzed temperature and humidity collected by the sensors and decided to issue an alert that was visualized on the "Application" in the user's smartphone.

Next, we analyze the activities centrality through the *eigenvector centrality* graph available in Prov-Dominoes for square matrices to measure the importance of the activities. The resulting graph is exhibited in Figure 17. The graph presents the activity "analyze_data" with the highest centrality score (stronger blue shade), suggesting it is the most important activity. Observing the graph together with the matrix of Figure 16.b, we can see the activities with higher scores are issued by the *agent* "Server", suggesting a dependence of an *agent* operating from the cloud.

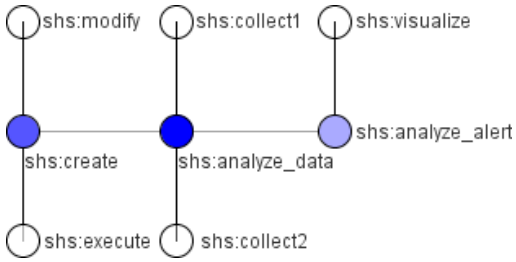


Figure 17: *Eigenvector centrality* graph on activities.

RQ1.1. *How Prov-Dominoes uncovers implicit information?*
In the provenance, there was no data direct relating activities (commands issued to devices) to each other. Such relation was made explicit by the communication inference: USD [Ac|E] × WGB [E|Ac] = WIB [Ac|Ac]. After uncovering activity communications, the centrality analysis of activities became possible.

RQ1.2. *How Prov-Dominoes provides a holistic perspective?*
The centrality graph in Figure 17 along with the matrices of Figure 16 assisted in providing a wholesome understanding of what the provenance of the smart home represents. Altogether, the analysis indicated how the collections from the sensors subsided a decision from an agent operating from a cloud ("Server") to issue an alert visualized on the user's smartphone.

RQ1.3. *How Prov-Dominoes supports concise analysis?*
The original provenance graph of Figure 14 has around four dozen boxes. In contrast, the centrality graph in Figure 17 has only eight nodes, from which it is possible to observe the collections from sensors ("collect1" and "collect2") and, together with the matrix of Figure 16.b, realize which agents are responsible for such activities, respectively: the sensors and the "Server". Altogether, such concise analysis needs fewer elements to be observed, yet retaining central information to the provenance understanding.

4.1.3. Exploratory Practices

This section discusses transformations and visualizations that articulated together during the explorations, working as agnostic **E**xploratory **P**ractices. Moreover, we summarize how such practices relate to the research questions of the effectiveness evaluation.

Exploring Combinations (EP1): We realized that exploring combinations involving provenance inferences, or just domino tiles targeted for analysis, allowed us to enrich domino tiles or derive new ones for all case studies by making implicit data explicit. As new data are uncovered, new exploration avenues unfold, contributing to a richer analysis. In particular, the following provenance inferences revealed themselves effective in unveiling implicit data:

- USD [Ac|E] × WGB [E|Ac] = WIB [Ac|Ac] (Cheney et al., 2013b) and
- WGB [E|Ac] × USD [Ac|E] = WDF [E|E] (Cheney et al., 2013e).

Centrality Analysis (EP2): Provenance graphs with many nodes and annotated elements such as the one in Figure 14 may overwhelm the user's capability to visually extract knowledge. The centrality analysis provides a per-type approach to analyze provenance data by visualizing reduced graphs such as the centrality graphs of activities, agents, or entities. Moreover, such a concise perspective provides indications of the importance of the nodes, as discussed in Section 3.2. For the **second case study**, we were able to identify important activities by employing centrality analysis.

Dependency Analysis (EP3): Dependency analysis is a useful technique that has many applications in software engineering and component-based systems (CBS) (Amalarethinam et al., 2015). For the **first case study**, where a series of components (modules) articulate together to perform some task, we employed the *transitive closure* operation to subside a dependency analysis. The analysis was capable of highlighting relevant parts of the workflow and providing an understanding of parameter influences.

Sparsity and Patterns Analysis (EP4): The matrix visualization itself provides an environment for sparsity and pattern analysis. The former can visu-

Table 2: Exploratory Practices addressing research questions of the effectiveness evaluation.

Case Study	**RQ1.1**	**RQ1.2**	**RQ1.3**
First (Workflow)	EP1, EP4	EP3, EP5	EP4
Second (Smart Home)	EP1	EP2	EP2, EP4

ally indicate the degree of matrix sparsity and provide an idea of data entropy. When done after sorting domino tiles, the latter may uncover patterns in rows and columns. Altogether, cell rearrangements and matrix visualization played an effective role in exploring the first two case studies, revealing patterns that assisted the analysis.

Dimension Reduction (EP5): For the first case study, we realized that removing empty rows/columns with the *Trim* operation reduced the matrix and facilitated pattern identification among cells. However, it is important to interpret the empty rows/columns before proceeding with such a reduction to avoid relevant information loss. An empty row or column is a pattern itself and may carry relevant meaning.

We summarize in Table 2 the exploratory practices observed in each case study for each research question of the effectiveness evaluation. We can see that **Exploring Combinations (EP1)** and **Sparsity and Patterns Analysis (EP4)** were employed in all case studies, to answer different research questions, suggesting them as effective exploratory practices for unveiling implicit data (**RQ1.1**) and supporting from concise analysis (**RQ1.3**), respectively.

4.2. Efficiency

In this section, we present the efficiency evaluation, where we process thousands of provenance relations in GPU and CPU to answer **RQ2**. First, we describe the materials and methods, and then we present and discuss the results.

4.2.1. Materials and Methods

The corpus of this evaluation consists of thousands of tweets (a post on Twitter) collected to measure how Prov-Dominoes performs on different processing modes and over increasing data volumes. We used a free subscription to the Twitter API to collect data, limiting the searches to seven days.

As the COVID-19 pandemic was a growing interest in social networks, we decided to collect tweets about Brazilian officials managing policies to mitigate the pandemic. The collected tweets subsided a sentiment analysis during the time frame from 05/20/2020 to 05/27/2020. We could collect more than four thousand tweets in this context.

When parsing the data to provenance, we identified the tweets as entities and the users as agents and publishing the tweets as activities. We categorized the users according to their influential scope (their followers) by using the "prov:type" attribute and the hashtags used in publishing were set as an *activity-type*. We used "::" as a separator to handle multiple hashtags:

```
activity(pub..., [prov:type = "#hashtag1::#hashtag2"]).
```

As a result, Prov-Dominoes generated five domino tiles: WAW [Ac|Ag], WGB [E|Ac], WAT [E|Ag], *activity-type* [Ac|T] (hashtags), and *agent-type* [Ag|T] (number of followers).

The explorations focused on an exploratory task described in an EPS available on the tool's website[14]. The exploratory task aimed at providing a perspective on the influential scope of users mentioning the Executive leaders. Initially, the EPS targeted only the full collection of 4,176 tweets both on CPU[15] and GPU[16]. Further, we decided to split the original collection into eight subsets of tweets, grouped according to their orders of magnitude[17] (m):

- Group 1 ($m = 2$): 100, 200, 300, and 500; and

[14] `https://tinyurl.com/papereps`.
[15] Intel Core i5 @ 2.90 GHz, 8 GB RAM.
[16] NVidia GeForce GT 1030, 2GB of dedicated RAM and 4GB of shared RAM.
[17] $N = a \times 10^b$, where $1 \leq a < 10$ and b is the order of magnitude.

- Group 2 ($m = 3$): 1000, 2000, 3000, and 4,176.

The reason for different collection sizes was to understand the curve behavior in different modes over increasing data volume as well as different orders of magnitude. Then, we organized a drill where we ran the EPS targeting each subset upwardly. For instance, a drill executes the EPS targeting subset 100, then 200, then 300, until subset 4,176, then we collect the aggregated time of the drill (sum of the timings of the executions of all subsets). As we were interested in evaluating the processing rather than the memory management, we ran the drill five times, eliminating extreme values and computing the average of the remaining three in order to mitigate the memory management impact. For all drills, we ran Prov-Dominoes in "auto-load" mode, where Prov-Dominoes executes the EPS after booting. Moreover, we set tuning and telemetry parameters to "true". The former avoids the optional matrix sorting before multiplication, and the latter outputs execution times on the JVM (Java Virtual Machine) console.

4.2.2. Results and Discussion

The combined results of the drills on CPU and GPU are depicted in Figure 18. The execution times are on a logarithmic scale. The numbers between the bars, rounded to the nearest integer, indicate the speedup (CPU time / GPU time) between the processing modes, representing how many times GPU was faster than CPU. GPU was approximately 127 times faster for the largest tweet collection than CPU. CPU was marginally (less than 100 milliseconds) faster for 100 and 200 tweets. In this case, GPU was slower because memory transfer adds time to the GPU execution. Such overhead is a common bottleneck between integrated CPU-GPU applications. From 300 tweets on, GPU becomes faster because the timing of the combinations and transformations in the CPU starts to exceed the memory transfer overhead. For 500 tweets, GPU already performs more than two times faster. For the complete dataset of 4,176 tweets, Prov-Dominoes runs in 5 seconds in GPU, still quite reasonable for exploratory analyses, while it would take 11 minutes in CPU. Such a time difference clearly

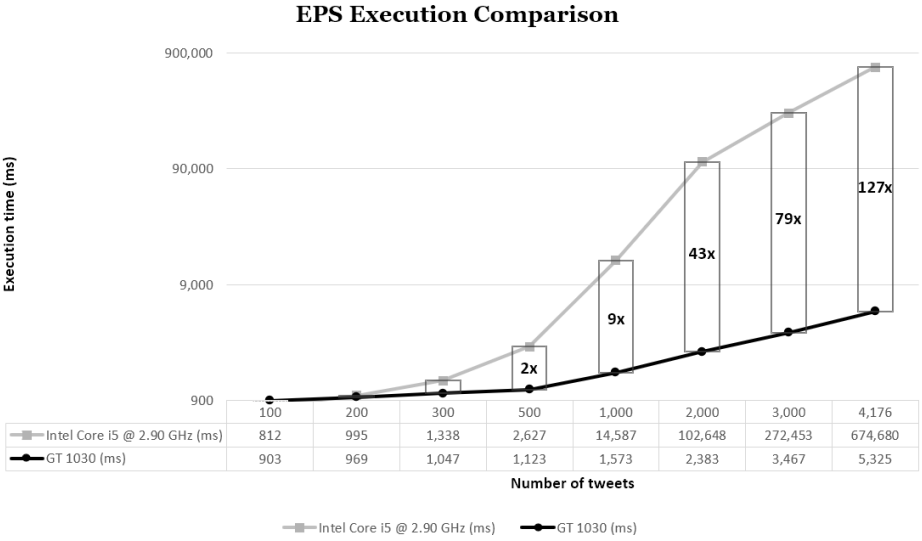


Figure 18: CPU-GPU Comparison with speedups.

shows that the GPU processing feature of Prov-Dominoes was paramount for allowing interactive explorations over large datasets, previously impracticable in CPU.

Going further, we discuss the results within magnitude groups for the individual commands used in Prov-Dominoes. We observe in Figure 19 stacked bars representing executions times of the commands in *EPS 2* for GPU and CPU side by side. For instance, GPU100 and CPU100 mean *EPS 2* executions targeting 100 tweets for GPU and CPU, respectively. We can see the LOAD command has the same size in CPU and GPU for the same number of tweets. This happens because the LOAD command is always performed in the CPU due to its dependence on I/O (file reading). The remaining commands are performed according to the processing mode. We can see that the multiply (top black bar) command takes more time as the number of tweets increases for CPU, up to the point of taking more than two times than the LOAD command for 500 tweets. Conversely, the multiply command remains more or less stable in GPU. Thus, for this order of magnitude (Group 1), we can observe that, in only one situation, a command surpasses the LOAD command in time: the multiply

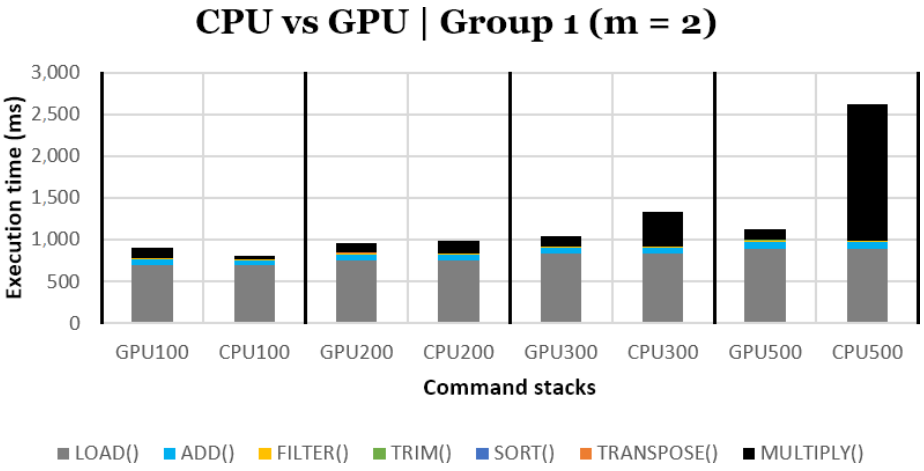


Figure 19: CPU-GPU Comparison (Group 1) on the commands time stack.

command running on the CPU for 500 tweets.

Considering Group 2, depicted in Figure 20, we can see that, altogether, the GPU executions remained more or less the same. Conversely, the multiply command in the CPU escalated overwhelmingly over the other commands.

> **RQ2**. *How efficient is Prov-Dominoes when running in GPU in comparison to CPU?*
>
> In our evaluation, from 300 tweets on, GPU was faster. Even for less than 300 tweets, the difference in favor of CPU was only marginal (less than 100 milliseconds), suggesting that the GPU processing mode fits any data volume. For larger data volumes, the speedups increased considerably, up to the point of approximately 127 times faster for the largest data volume (4,176 tweets).

4.3. Threats to Validity

Besides our efforts to avoid limitations to our study, as in every evaluation, we are subject to some threats to validity, as discussed in the following.

Internal. Two internal threats stand out: (1) during the efficiency evaluation, we could have made the CPU implementations faster by implementing them in C or assembly. However, the results in favor of GPU were so compelling that we refrained from duplicating the Java implementations on faster

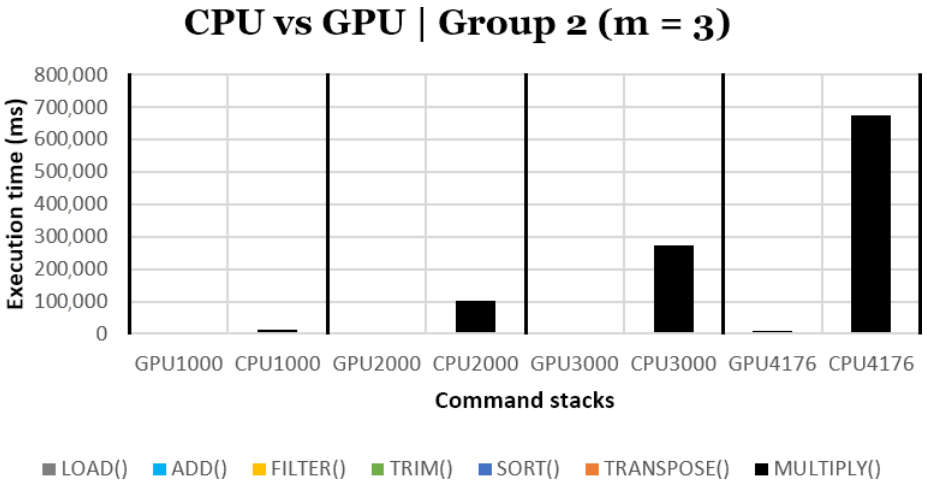


Figure 20: CPU-GPU Comparison (Group 2) on the commands time stack.

languages; and (2) we only considered one opinion on the first case study (Workflow). Although this opinion may be biased, we tried to mitigate it by seeking for the opinion of the workflow's author. Finally, the results from the Smart Home case studies were not validated by specialists. We tried to validate the results by trying to contact the author of the provenance data several times, with no success. Nonetheless, we opted to keep this case study in this article. Our primary objective in doing so was to elucidate certain facets of the Exploratory Practices as delineated in Section 4.1.1. It is worth noting that we conducted a separate investigation (da Silva Junior et al., 2022) involving interviews with professionals from the industry. This study aimed to unravel the ways in which our proposed approach contributed to their ability to address pertinent questions in their daily professional endeavors.

Construct. During the effectiveness evaluation, we did not always stick to concrete measures. While addressing the research question **RQ1.2** (*How Prov-Dominoes provides a holistic perspective?*), we did it in an interpretive way, relying on an abstract notion of the meaning of holistic. However, we tried to mitigate such a threat to **RQ1** by considering concrete measures for the remaining two sub-questions by identifying explicit data for **RQ1.1** and

patterns and graph comparisons in **RQ1.3**.

External. We did not account for how the tool would be operated by other users rather than the authors. New users may experience steep learning curves while exploring the tool. Nevertheless, as Prov-Dominoes uses the same user interface introduced by Dominoes, we believe that our previous user evaluation study (da Silva Junior et al., 2022) performed with the latter could help to mitigate such a threat. This evaluation has been performed with real participants for usability, learning curve, and usefulness, among other aspects. In Dominoes, the domain is software development history, and dominoes tiles represent developers, commits, altered files, etc. We used a scenario-based, think-aloud study, where we presented participants with situations that they had to investigate a project's history using Dominoes to answer a set of questions. In this study, we reached a learning curve result of 15 minutes, achieving a success rate of 86% in their explorations. Some impressions provided by the study participants include *"I am impressed... Dominoes produces data very fast"*. Finally, most participants addressed Dominoes as an efficient, time-saving, fast, innovative, and useful tool[18]. Additionally, the exploratory practices identified after our explorations on the effectiveness evaluation attempt to mitigate such learning curves by suggesting practices that may guide their explorations. Conversely, the exploratory practices may not be as assistful as they were due to the reduced number of considered case studies. Although we used only three case studies for the effectiveness assessment, we tried to mitigate such a threat by considering distinct domains.

Conclusion. The memory transfer bottleneck discussed in Section 4.2.2 is a sensitive matter while measuring the timing of the executions. Such sensitiveness happens because memory management has different architectural organizations between CPU and GPU, leading to different management strategies. When performing the same operation repeatedly and taking the average time,

[18]To capture these impressions, we used Microsoft Reaction Cards, Developed by ®. 2002 Microsoft Corporation. All rights reserved.

the results may vary significantly. The memory management impact can contribute to either decreasing or increasing the execution time.

5. Related Work

We grouped our related work into two categories: visualization-based and exploratory analysis provenance tools. We highlight two subgroups inside the visualization group: domain-specific and W3C PROV compatible tools, adequate to represent more domains. In the exploratory analysis group, provenance-based Workflow Management Systems (WfMSs) arose, providing infrastructure for data exploration and visualization through workflows.

Regarding visualization-based tools, GENI (Chen et al., 2012) provides visualization of network data provenance and InProv (Borkin et al., 2013) provides visualizations of file system provenance data. As domain-specific tools, they individually have some interesting features, such as interactive radial-based tree layout, abstract views, breadth-first searches, and graph comparison. ProvenanceMatrix (Dang et al., 2015) is a tool specific for taxonomies, aiding users to align and understand the relationships between different taxonomies and how they relate to each other. Using a matrix for visualization, it provides operations to filter, order, link elements, and collapse/expand sub-hierarchies. However, as visualization-centered tools, they do not provide data combinations as Prov-Dominoes. Moreover, these features are not available in an integrated way, hindering the analysis due to visualization and manipulation restrictions. Distinct analyses in Prov-Dominoes can be integrated through dominoes tiles combinations for different domains.

Outside of the domain-specific visualization tools and using heterogeneous data sources, Domino (Gratzl et al., 2014) allows users to combine and visualize information for multiple interconnected datasets. A set of operations for partitioning, sorting, and grouping such data are available. It also uses the concept of dominoes pieces for representing data and available connections, providing a wide range of visualizations. However, Domino aims to work with tabular data

such as vectors and matrices. In contrast, Prov-Dominoes allows analysis over graphs, which is a non-structured type of data. Besides that, Prov-Dominoes allows different operations to be performed over, such data such as pass filters, eigenvector centrality, and transitive closure, which can aid analysis exploration for implicit relationships. For related work using provenance domain, we can cite some general-purpose visualization tools that are compatible with W3C PROV, such as ProvToolbox (Moreau, 2016), Provenance Explorer (Cheung & Hunter, 2006), and Prov Viewer (Kohwalter et al., 2016). ProvToolbox was one of the first W3C PROV compatible tools, converting the PROV-DM representations into various formats. However, it lacks a built-in visualization and requires a generic graph tool to visualize the provenance data. Provenance Explorer takes RDF-based provenance outputs from capture systems and dynamically generates customized views of the provenance trail. Prov Viewer enables users to explore provenance data over graphs interactively. Different from Provenance Explorer, Prov Viewer provides data combinations through graph merges. Despite being graph-based, neither tools provide score-based centrality analysis as Prov-Dominoes' *eigenvector centrality*. Despite their visualization features, such tools are sensitive to large data, overwhelming the user's ability to extract knowledge visually. On the other hand, Prov-Dominoes' matrix visualization can represent hundreds or thousands of graph nodes coupled with guiding features, such as row/column patterns unveiled by matrix sorting and data dispersion through *Z-Score* operation.

Two WfMSs are widely known for supporting scientific experiments: Taverna (Hull et al., 2006) and VisTrails (Callahan et al., 2006). Taverna is an application that eases the use and integration of the growing number of molecular biology tools and databases available on the web. It allows bioinformaticians to construct workflows or pipelines of services to perform various analyses, such as sequence analysis and genome annotation. VisTrails allows users to navigate workflow versions intuitively, visually compare different workflows and their results, and examine the actions that led to a result. Different from Taverna, VisTrails provides provenance of workflow evolution and has built-in provenance

capture of workflow executions. Taverna needs plugins for provenance capture of workflow executions. Despite their concise visualization through workflows, they hide parameters used by modules in the workflow view and lack features to analyze module parameters' influences, needing to delegate such analysis to other provenance tools.

Oliveira et al. (2018) provide a comprehensive collection of work related to provenance analysis. They propose a taxonomy that classifies provenance analysis papers and discusses several approaches under the Access Methods/Visual category. From those, we highlight ProvAbs (Missier et al., 2014), ProvOwl (Karsai et al., 2016), and AVOCADO (Stitz et al., 2016). ProvAbs defines a Provenance Abstraction Model to store PROV-N scripts in a Neo4J database, where queries can be posed by using the Neo4J Traverse API and Cypher language. ProvOwl represents provenance as OWL graphs and allows users to combine several nodes, zoom in/out, filter, or rearrange nodes. AVOCADO (Adaptive Visualization of Comprehensive Analytical Data Origins) is an approach to reduce the complexity of the provenance graph using hierarchical and motif-based aggregation. Despite their relevant contributions, none of these works addresses the problems we discuss in this paper.

Visionary (Oliveira et al., 2022) is a framework designed to assist in the understanding and use of provenance data through ontologies. It captures the provenance data and generates new information using Prov-O (Lebo et al., 2013) and structural analysis of the provenance graph. The visualization presents and highlights inferences and results obtained with the data analysis. The tool primarily leverages the influence relations of Prov-DM to generate inferences and enrich the original graph. In contrast, Prov-Dominoes delves deeper into Prov-DM, harnessing a broader range of relations, potentially offering a more comprehensive view of the data. While Prov-Dominoes offers a detailed exploration of provenance data, Visionary's strength lies in its ability to seamlessly integrate with other ontology-based systems, enhancing its utility in diverse environments that prioritize generic model representations.

Our choice of using matrices to perform data exploration analysis is backed

up by several authors in the literature (Burch et al., 2021; Ghoniem et al., 2005; Elmqvist et al., 2008). Using graphs to manage large amounts of data can be challenging in terms of visualization and exploration. In fact, trying to follow individual anomalies or outliers in a node-link diagram can be a problem for large graphs. Even when different layout algorithms are used for data clustering, some of these clusters can be occluded by dense regions. Adjacency matrices can solve occlusion problems by allowing thousands of vertices and their connection to be represented clearly. Besides that, depending on the ordering strategies used on such matrices, it can unveil important patterns that could be difficult to observe in a graph-based visualization. Following paths in an adjacency matrix-based visualization can be minimized by employing a reordering approach (Burch et al., 2021).

Ghoniem et al. (2005) conducted an interesting study by comparing node-link and matrix-based representation for graph visualization. By using graph sizes of up to 100, they concluded that matrices are better suitable for large or dense graphs, but that path-related tasks are difficult in both representations. For processing those large matrices, ZAME (Elmqvist et al., 2008) uses GPU instead of CPU and shows a significant performance improvement in graph renderization. This motivates us to use GPU to implement the matrix operations in ProvDominoes.

6. Conclusion and Future Work

Although provenance graph visualization is common, such support is limited when there is a need to combine data to uncover implicit information. Moreover, in complex domains, the provenance graphs can be composed of thousands of vertices and edges, becoming visually cluttered and limiting the user's ability to visually and interactively analyze the data. Prov-Dominoes addresses such challenges by providing concise visualizations through matrices and reduced centrality graphs, besides offering features and practices to combine, transform, and reduce data, unveiling implicit information and contributing to a more

comprehensive perspective of the data domain.

Our tool is compatible with PROV-N notation, allowing its adoption in different domains and applications. Furthermore, it can enrich provenance data with few explicit expressions, applying provenance inferences to uncover new analysis possibilities usually neglected due to rich information buried in the data. Although Prov-Dominoes primarily focuses on the PROV-N format, it can also process data models that are compatible with matrix formats. For this functionality, it is essential that the target data model be converted into matrices following the MATRIX format[19]. Subsequently, these matrices can be imported into Prov-Dominoes through its main menu. This procedure allows the processed data to be used as domino tiles. As a result, it can benefit from all operations and visualizations that the tool offers. This approach significantly expands the applicability of Prov-Dominoes, enabling the analysis of a broader range of datasets.

We presented three case studies from different domains and identified exploratory guidelines for practical analysis. Thus, we could uncover implicit information, such as influencing parameters in a workflow execution, and provide wholesome perspectives, such as the main components of a workflow and essential activities in a smart home. Additionally, the efficiency evaluation in the context of Twitter sentiment analysis revealed that GPU was 127 times faster in executions involving combinations of thousands of relations.

Future work includes implementing more algorithms for clustering and sorting and extending GPU implementations for the vast majority of matrix transformations. Finally, as the tool provides means to provenance inferences, it could be adapted to work as an environment for producing normal forms of PROV instances and assist in checking PROV validity and equivalence properties (Cheney et al., 2013f).

[19] `https://tinyurl.com/MATRIXFormat`

7. Acknowledgement

The authors would like to thank CASNAV (Centro de Análises de Sistemas Navais), CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico), and FAPERJ (Fundação de Amparo à Pesquisa do Estado do Rio de Janeiro) for the financial support.

References

Ackerman, M. J. (1998). The visible human project. *Proceedings of the IEEE*, *86*.

Amalarethinam, G., Maitheen, P., & Hameed, S. (2015). Dependency analysis for component based systems using graphs. *International Journal of Applied Engineering Research*, *10*.

Bastian, M., Heymann, S., Jacomy, M. et al. (2009). Gephi: an open source software for exploring and manipulating networks. *Icwsm*, *8*, 361–362.

Belhajjame, K., B'Far, R. et al. (2013). Prov-DM: The PROV Data Model. URL: `http://www.w3.org/TR/prov-dm` W3C Recommendation.

Berners-Lee, T., Jaffe, J. et al. (1994). World Wide Web Consortium (W3C). URL: `https://www.w3.org/`.

Borkin, M. A., Yeh, C. S., Boyd, M., Macko, P., Gajos, K. Z., Seltzer, M., & Pfister, H. (2013). Evaluation of filesystem provenance visualization tools. *IEEE Transactions on Visualization and Computer Graphics*, *19*, 2476–2485.

Bouch, A., Kuchinsky, A., & Bhatti, N. (2000). Quality is in the eye of the beholder: Meeting users' requirements for internet quality of service. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* CHI '00 (p. 297–304). New York, NY, USA: Association for Computing Machinery. URL: `https://doi.org/10.1145/332040.332447`. doi:`10.1145/332040.332447`.

Burch, M., ten Brinke, K. B., Castella, A., Peters, G. K. S., Shteriyanov, V., & Vlasvinkel, R. (2021). Dynamic graph exploration by interactively linked node-link diagrams and matrix visualizations. *Visual Computing for Industry, Biomedicine, and Art*, *4*, 23. URL: https://doi.org/10.1186/s42492-021-00088-8. doi:10.1186/s42492-021-00088-8.

Callahan, S. P., Freire, J., Santos, E., Scheidegger, C. E., Silva, C. T., & Vo, H. T. (2006). Vistrails: visualization meets data management. In *Proceedings of the 2006 ACM SIGMOD international conference on Management of data* (pp. 745–747).

Chen, P., Plale, B., Cheah, Y.-W., Ghoshal, D., Jensen, S., & Luo, Y. (2012). Visualization of network data provenance. In *2012 19th International Conference on High Performance Computing* (pp. 1–9). IEEE.

Cheney, J., Missier, P., & Moreau, L. (2013a). Attribution Inference. URL: https://www.w3.org/TR/prov-constraints/#attribution-inference Constraints of the PROV Data Model. W3C Recommendation.

Cheney, J., Missier, P., & Moreau, L. (2013b). Communication Inference. URL: https://www.w3.org/TR/prov-constraints/#generation-use-communication-inference_text Constraints of the PROV Data Model. W3C Recommendation.

Cheney, J., Missier, P., & Moreau, L. (2013c). Constraints of the PROV Data Model. URL: https://www.w3.org/TR/prov-constraints W3C Recommendation.

Cheney, J., Missier, P., & Moreau, L. (2013d). Delegation Inference. URL: https://www.w3.org/TR/prov-constraints/#delegation-inference Constraints of the PROV Data Model. W3C Recommendation.

Cheney, J., Missier, P., & Moreau, L. (2013e). Derivation Inference. URL: https://www.w3.org/TR/prov-constraints/

#derivation-generation-use-inference Constraints of the PROV Data Model. W3C Recommendation.

Cheney, J., Missier, P., & Moreau, L. (2013f). Normalization, validity, and equivalence. URL: https://www.w3.org/TR/prov-constraints/#normalization-validity-equivalence Constraints of the PROV Data Model. W3C Recommendation.

Cheney, J., Missier, P., Moreau, L., & Soiland-Reyes, S. (2013g). PROV-N: The provenance notation. URL: https://www.w3.org/TR/prov-n/ W3C Recommendation.

Cheung, K., & Hunter, J. (2006). Provenance explorer–customized provenance views using semantic inferencing. In *International Semantic Web Conference* (pp. 215–227). Springer.

Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on software engineering*, *20*, 476–493.

Closa, G., Masó, J., Proß, B., & Pons, X. (2017). W3c prov to describe provenance at the dataset, feature and attribute levels in a distributed environment. *Computers, Environment and Urban Systems*, *64*, 103–117.

Committee, P. E. et al. (2008). Premis data dictionary for preservation metadata. *Preservation*, .

Crucitti, P., Latora, V., & Porta, S. (2006). Centrality in networks of urban streets, chaos. *Quarterly of the American Institute of Physics*, *16*.

Dang, T., Franz, N. M., Ludäscher, B., & Forbes, A. G. (2015). ProvenanceMatrix: A visualization tool for multi-taxonomy alignments. In V. Ivanova, P. Lambrix, S. Lohmann, & C. Pesquita (Eds.), *Proceedings of the International Workshop on Visualizations and User Interfaces for Ontologies and Linked Data co-located with 14th International Semantic Web Conference (ISWC 2015), Bethlehem, Pennsylvania, USA, October 11, 2015* (p. 13). CEUR-WS.org volume 1456 of *CEUR Workshop Proceedings*.

Del Rio, N., & Da Silva, P. P. (2007). Probe-it! visualization support for provenance. In *International Symposium on Visual Computing* (pp. 732–741). Springer.

Ellson, J., Gansner, E. R., Koutsofios, E., North, S. C., & Woodhull, G. (2004). Graphviz and dynagraph—static and dynamic graph drawing tools. In *Graph drawing software* (pp. 127–148). Springer.

Elmqvist, N., Do, T.-N., Goodell, H., Henry, N., & Fekete, J.-D. (2008). Zame: Interactive large-scale graph visualization. In *2008 IEEE Pacific Visualization Symposium* (pp. 215–222). doi:10.1109/PACIFICVIS.2008.4475479.

Freire, J., Koop, D., Santos, E., & Silva, C. T. (2008). Provenance for computational tasks: A survey. *Computing in Science & Engineering*, *10*, 11–21.

Ghoniem, M., Fekete, J.-D., & Castagliola, P. (2005). On the readability of graphs using node-link and matrix-based representations: A controlled experiment and statistical analysis. *Information Visualization*, *4*, 114–135. URL: https://doi.org/10.1057/palgrave.ivs.9500092. doi:10.1057/palgrave.ivs.9500092.

Gratzl, S., Gehlenborg, N., Lex, A., Pfister, H., & Streit, M. (2014). Domino: Extracting, comparing, and manipulating subsets across multiple tabular datasets. *IEEE Trans. Vis. Comput. Graph.*, *20*, 2023–2032.

Groth, P., Moreau, L., Missier, P., Freire, J. et al. (2006). International Provenance and Annotation Workshop (IPAW). URL: http://ipaw.info/.

Hull, D., Wolstencroft, K., Stevens, R., Goble, C., Pocock, M. R., Li, P., & Oinn, T. (2006). Taverna: a tool for building and running workflows of services. *Nucleic acids research*, *34*, W729–W732.

Karsai, L., Fekete, A., Kay, J., & Missier, P. (2016). Clustering provenance facilitating provenance exploration through data abstraction. In *Proceedings of the Workshop on Human-In-the-Loop Data Analytics* (pp. 1–5).

Kohwalter, T., Oliveira, T., Freire, J., Clua, E., & Murta, L. (2016). Prov viewer: A graph-based visualization tool for interactive exploration of provenance data. In *International Provenance and Annotation Workshop* (pp. 71–82). Springer.

Lebo, T., Sahoo, S., Mcguinness, D., Belhajjame, K., Cheney, J., Corsar, D., Garijo, D., Soiland-Reyes, S., Zednik, S., & Zhao, J. (2013). Prov-o: the prov ontology. w3c recommendation. URL: `https://www.w3.org/TR/prov-o/` w3C Recommendation.

Macko, P., & Seltzer, M. (2011). Provenance map orbiter: Interactive exploration of large provenance graphs. In *TaPP* (pp. 1–6).

Missier, P., Bryans, J., Gamble, C., Curcin, V., & Danger, R. (2014). Provabs: model, policy, and tooling for abstracting prov graphs. In *International Provenance and Annotation Workshop* (pp. 3–15). Springer.

Missier, P., & Moreau, L. (2013). PROV-Overview. An overview of the PROV family of documents. URL: `https://www.w3.org/TR/prov-overview/` W3C Working Group Note.

Missier, P., Moreau, L. et al. (2013). Prov-dm relations at a glance. URL: `https://www.w3.org/TR/2013/REC-prov-dm-20130430/#relations-at-a-glance`.

Moreau, L. (2016). Provtoolbox - java library to create and convert w3c prov data model representations. [Online]. Available: http://lucmoreau.github.io/ProvToolbox/.

Moreau, L., Clifford, B., Freire, J., Futrelle, J., Gil, Y., Groth, P., Kwasnikowska, N., Miles, S., Missier, P., Myers, J. et al. (2011). The open provenance model core specification (v1. 1). *Future generation computer systems*, *27*, 743–756.

Moreau, L., Freire, J., Futrelle, J., Mcgrath, R. E., Myers, J., & Paulson, P. (2008a). The open provenance model: An overview. In *International Provenance and Annotation Workshop* (pp. 323–326). Springer.

Moreau, L., Ludäscher, B., Altintas, I., Barga, R. S., Bowers, S., Callahan, S., Chin Jr, G., Clifford, B., Cohen, S., Cohen-Boulakia, S. et al. (2008b). The first provenance challenge. *Concurrency and computation: practice and experience*, *20*, 409–418.

Newbury, D., & Lippincott, L. (2019). Provenance in 2050. In J. Milosch, & N. Pearce (Eds.), *Collecting and Provenance: A Multidisciplinary Approach* chapter 7. (p. 104). Rowman & Littlefield Publishersr. (1st ed.).

Newman, M. E. (2008). The mathematics of networks. *The new palgrave encyclopedia of economics*, *2*, 1–12.

Oliveira, W., Oliveira, D. D., & Braganholo, V. (2018). Provenance analytics for workflow-based computational experiments: A survey. *ACM Computing Surveys*, *51*.

Oliveira, W. d., Braga, R., David, J. M. N., Ströele, V., Campos, F., & Castro, G. (2022). Visionary: a framework for analysis and visualization of provenance data. *Knowledge and Information Systems*, . doi:10.1007/s10115-021-01645-6.

Schroeder, W., Martin, K., & Lorensen, B. (2006). *Visualization Toolkit: An Object-Oriented Approach to 3D Graphics.*. Kitware. URL: https://vtk.org/vtk-textbook/.

Silva, C. T., Freire, J., & Callahan, S. P. (2007). Provenance for visualizations: Reproducibility and beyond. *Computing in Science & Engineering*, *9*, 82–89.

da Silva Junior, J. R., Campagna, D. P., Clua, E., Sarma, A., & Murta, L. G. P. (2022). Dominoes: An interactive exploratory data analysis tool for software relationships. *IEEE Transactions on Software Engineering*, (pp. 377 – 396).

Stitz, H., Luger, S., Streit, M., & Gehlenborg, N. (2016). Avocado: Visualization of workflow–derived data provenance for reproducible biomedical research. In *Computer Graphics Forum* (pp. 481–490). Wiley Online Library volume 35.

Wu, H.-M., Tzeng, S., & Chen, C.-h. (2008). Matrix visualization. In *Handbook of data visualization* (pp. 681–708). Springer.

Yadav, A., & Khan, R. (2010). Does coupling really affect complexity? In *2010 International Conference on Computer and Communication Technology (ICCCT)* (pp. 583–588). IEEE.

Zaki, M. J., Meira Jr, W., & Meira, W. (2014). *Data mining and analysis: fundamental concepts and algorithms*. Cambridge University Press.